CNN Depth and Hyperparameter Effects on CIFAR-10

Gabriel Lopes
UC San Diego

Abstract

This project studies the effect of architectural depth and hyperparameters of several training setups for CNNs classifying images from the CIFAR-10 dataset. A very flexible variant of a common CNN architecture has been implemented with tunable depth, learning rate and choice of optimizers, as well as with options for different pooling schemes, activations and number of training iterations. We explore the performance of CNNs of depth from 2 to 4 convolutional blocks under several different combinations of the parameters of the flexible variant, training several models of each configuration and presenting results for each against the test set, with measurement given as the test classification accuracy. A variety of experiments were carried out and the results were summarized. It is seen that increasing the depth of a network will generally increase the accuracy of the network as long as the training is carried out for sufficient number of epochs. We compared Adam and stochastic gradient descent and found similar performances. We also found that moderate learning rates give better results for optimization. Overall, the experiments demonstrate that both architectural design and training strategy significantly influence CNN accuracy on standard image classification benchmarks such as CIFAR-10.

1 Introduction

Image classification is arguably the most widely used method for classification tasks in computer vision and it's used in many real-world applications such as self-driving vehicles and robotics, medical imaging and even in some security systems. Learning hierarchical spatial features directly from images by exploiting local connections and shared weights is what exactly CNNs do in order to classify images. The famous Image classification contest run by ImageNet shows this clearly. The pioneer network of this contest was LeNet-5 which simply demonstrated the fact that some patterns can be effectively detected with the help of convolutional layers. Then with AlexNet and more clever optimization techniques followed with Residual Networks, it is finally shown that depth in a network has significant impact on the network's classification accuracy and that much deeper networks can be actually trained with adequate measures such as the vanishing gradient problem.

Although significant strides have been made in the fields of architectural and training design, it is unclear how they actually bear down on any given problem. Experimental results are highly dependent on a wide range of factors. These include the depth of the network, choice of optimizer, learning rate, pooling method, choice of activation functions, etc. On moderate-sized datasets such as the benchmark CIFAR-10 dataset, it is not entirely clear which of these factors has the largest impact on the resulting classification accuracy. This research investigates the effect of architectural depth and training hyperparameters on the performance of CNNs on the CIFAR-10 image classification task.

This question is investigated by designing a CNN for which we can experiment with the various design parameters. Hence, we experiment with the number of convolutional layers in the network, different optimizers which are the Adam and stochastic gradient descent optimizers, effect of varying learning rates on the performance of network, by replacing max pooling and by evaluating different activation functions such as ReLU and Leaky ReLU within the convolutional blocks, and at last we see what effect increasing the training period has on network performance on CIFAR-10 dataset.

2 Methodology

2.1 CNN architecture and training procedure

We compare various convolutional neural network architectures and optimization strategies for the image classification task on the CIFAR-10 data set. All experiments were implemented using the PyTorch deep learning framework and based on mini-batch stochastic gradient descent for training. The model parameters were initialized randomly and new mini-batches were randomly shuffled at the start of each training epoch to improve training stability. Rather than using some form of automated search procedure, hyperparameter values are selected through a systematic empirical comparison. We evaluate models using classification accuracy on a fixed held-out test set, with experiments using a variety of different training durations to investigate the relationships between convergence, the choice of hyperparameters, and generalization.

2.1.1 Convolutional feature extractor

The proposed CNN employs a modular convolutional block design similar to that of the pioneering classical network architectures such as LeNet and later on in deep CNN models. The basic structure of a convolutional block consists of a convolution layer followed by possibly batch normalization, a non-linear activation function and spatial pooling. Channel dimensionality increases step-by-step across blocks following the structure: $3 \rightarrow 32 \rightarrow 64 \rightarrow 128 \rightarrow 256$. The deeper layers learn increasingly abstract representations. In between the blocks spatial dimensions are decreased by a factor of 2 by using max pooling, which gives a higher processing rate and increases the size of the receptive field.

2.1.2 Fully connected classification head

After the convolutional feature extraction layer, the corresponding feature maps are flattened and fed into a fully connected layer with 256 output dimension. Dropout with a drop probability of 0.3 is added after fully connected layer to prevent overfitting. Then the output of fully connected layer is fed into a final linear layer which is the output layer of classification and predicts scores for each of the 10 classes of CIFAR-10.

2.1.3 Configurable architectural and optimization parameters

The network allows controlled variations in several design parameters. Examples of design parameter variations which have been used in this work are the number of convolutional layers in the feature extraction layers (typically between 2 and 4 layers) and the choice of activation function, between ReLU and Leaky ReLU. Other design parameter variations are the choice of pooling operation (max or average pooling) and whether batch normalization is used or not. Our optimization strategy was varied by comparing stochastic gradient descent and Adam as optimizers, assessing the learning rate sensitivity of the model by trying out several configurations and adjusting the length of training to check for convergence. The experimental approach consisted of a systematic exploration of different configurations in order to study in details the effect of the depth of the network and the choice of hyperparameters on the accuracy of the final classifier.

2.1.4 Training details

Trained on mini-batches of size 128. Normalized by standard CIFAR-10 channel means when training with stochastic gradient descent and using momentum to help with the convergence. Number of training epochs was selected for each experiment. Figures shown in the early epochs may be a consequence of models that over-train on the small numbers of epochs shown. Cross-entropy loss is used as the optimization objective. Model checkpoints are stored and evaluated at the end of each epoch to monitor the performance on validation set and the stability of training.

2.1.5 Experimental protocol

Experiments were carried out by training several models in succession, with slightly different parameters each time, and comparing their performance with each other on the CIFAR-10 test set based on their classification accuracy. Different network depths are explored by varying the number of convolutional blocks by 2, 3 and 4. We provide further comparisons within each of the considered architectural settings across activation functions, pooling methods, optimizers and learning rates. We also investigate the impact of increasing the number of training iterations. We picked the best settings in each category based on the accuracy on the final test to make sure that we compared the same number of layers and the same training methodology.

2.1.6 Implementation details

All experiments were done using PyTorch with CPU computations. The default weight initialization of the framework was used for all the layers. We did not apply any data augmentation to isolate the effects of different architectures and hyperparameters. Increasing the depth of the networks caused longer training times and, in some cases, we extended the training duration in order to understand the full convergence and generalization capabilities of the models.

2.2 Dataset

For this comparison, we used the CIFAR-10 dataset. It contains 60,000 images of size 32x32 pixels that belong to 10 classes such as airplane, automobile, bird, cat, deer, dog, frog, horse, ship and truck. There are 50,000 training images and 10,000 testing images. We applied the standard preprocessing and normalization.

3 Experimental Results

Tables and figures report test accuracy under different architectural depths and training configurations on CIFAR-10.

Baseline architectural and optimization configurations evaluated after 10 training epochs:

Model	Optimizer	BN	Activation	Pool	Test Accuracy (%)
2-block CNN	Adam	No	ReLU	Max	72.12
2-block CNN	SGD	No	ReLU	Max	**72.86**
2-block CNN	Adam	Yes	ReLU	Max	71.70
2-block CNN	Adam	Yes	Leaky	Max	72.53
2-block CNN	Adam	Yes	ReLU	Avg	71.30

Effect of CNN depth:

Depth	Epochs	Test Accuracy (%)
2 blocks	10	72.86
3 blocks	10	77.66
4 blocks	10	79.94
3 blocks	30	82.62
4 blocks	30	**85.17**

Figure 1 shows the best test accuracy obtained by CNN models with different architectural depths.

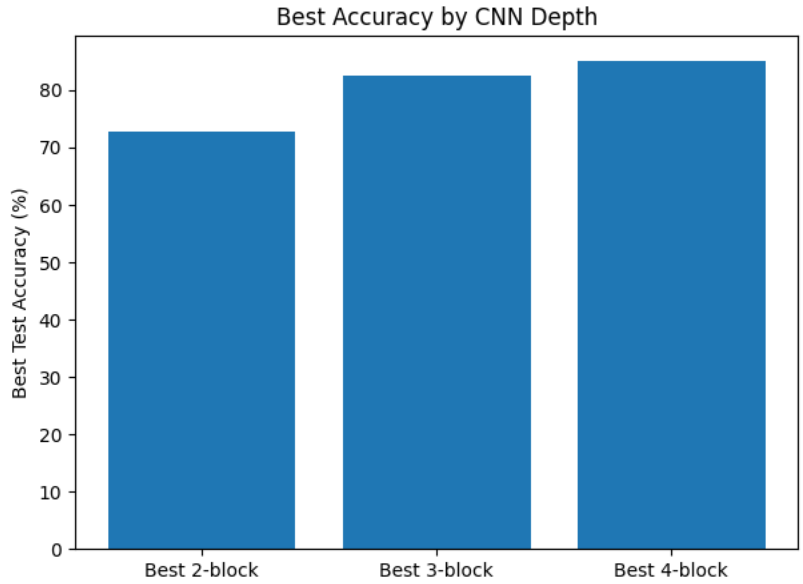

Figure 1. Best test accuracy achieved by CNN architectures with different depths.

Figure 2 presents test accuracy progression across training epochs for 3-block and 4-block CNN models.

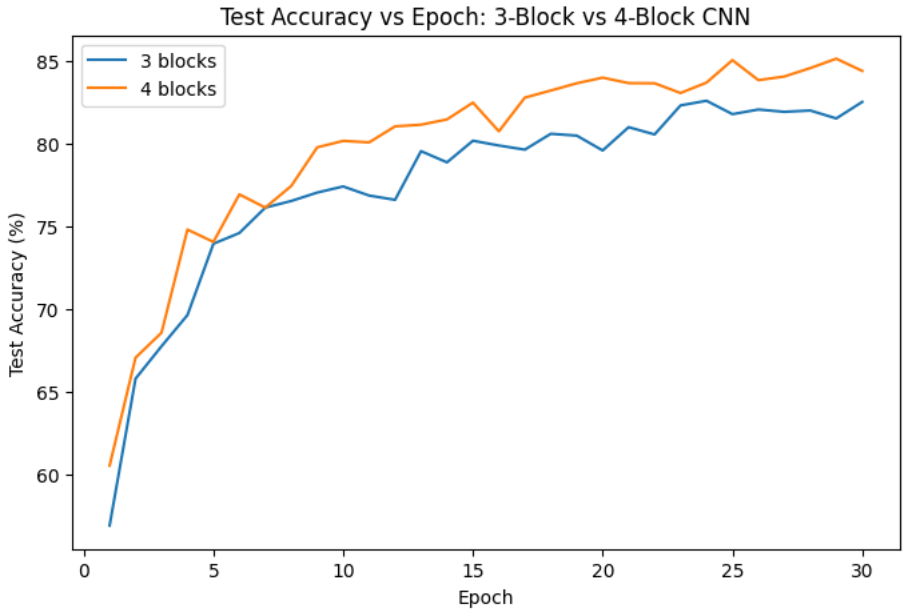

Figure 2. Test accuracy progression across training epochs for 3-block and 4-block CNN models.

Learning rate comparison (4-block CNN):

Learning Rate	Best Accuracy (%)
0.0005	84.37
0.001	**85.17**
0.002	85.03

Figure 3 illustrates the effect of different learning rates on training performance for the 4-block CNN.

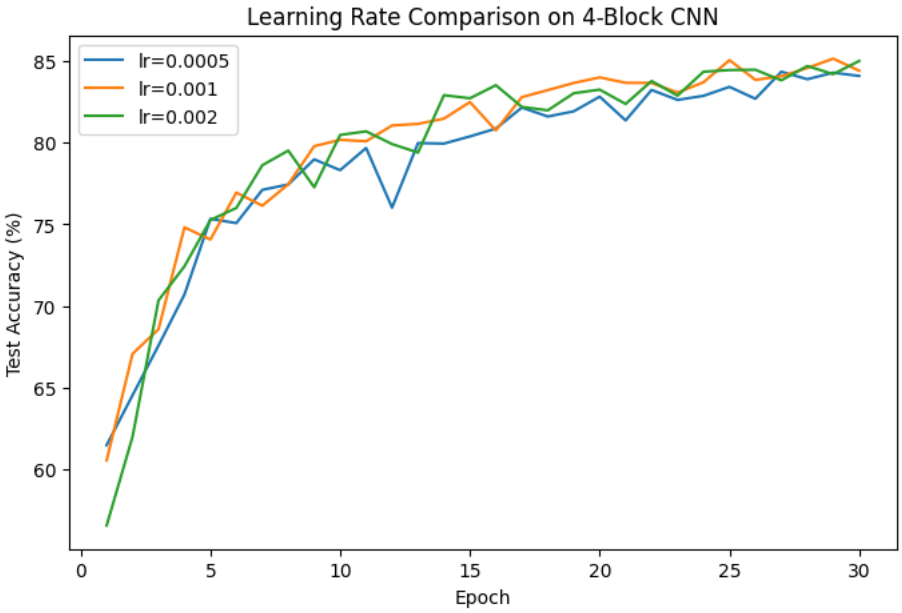

Figure 3. Effect of learning rate on test accuracy during training for the 4-block CNN.

Optimizer comparison (4-block CNN):

Optimizer	Best Accuracy (%)
Adam	**85.17**
SGD	84.72

Figure 4 compares training dynamics when using Adam and SGD optimizers for the 4-block CNN.

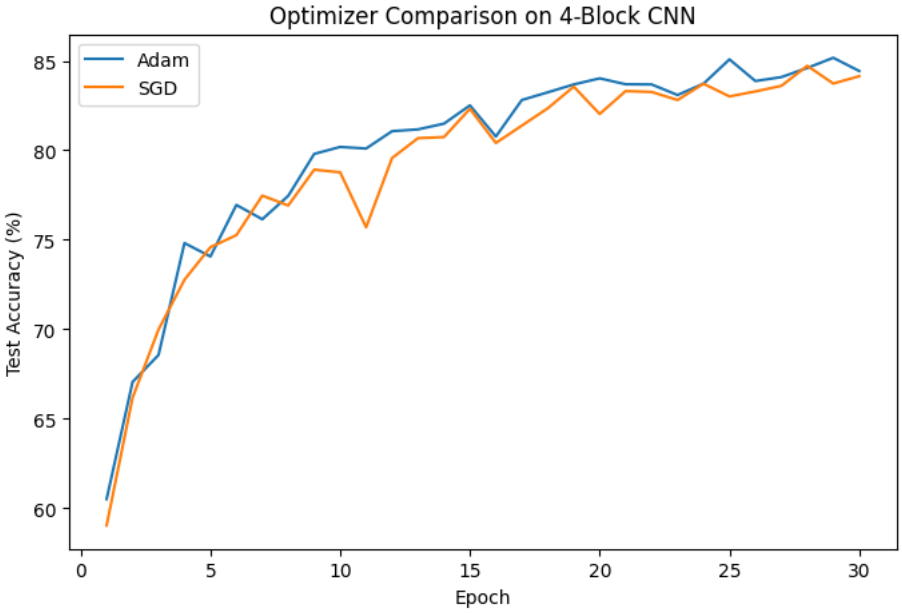

Figure 4. Comparison of training dynamics between Adam and SGD optimizers for the 4-block CNN.

Best performing CNN configuration identified in experiments:

Depth	Optimizer	LR	BN	Activation	Pool	Epochs	Best Accuracy (%)
4	Adam	0.001	Yes	ReLU	Max	30	**85.17**

Bold values indicate the best performance within the same comparison group.

4 Discussion and conclusion

4.1 Effect of architectural depth

Based on our experimental results we can see that the accuracy is improving as the depth of the network is increasing. Deep networks of depth 3 and 4 were shown to outperform the shallow network of depth 2 on the given test set. This is consistent with the fact that deeper networks are able to capture the higher level abstraction of the data which leads to a better discrimination between the different classes in the CIFAR-10 set. However, deeper models do not always yield higher accuracy in our experimental setup. Increasing network depth improved test accuracy, but deeper models benefited more from extended training to achieve their best performance. Even though our deepest models required more epochs to reach convergence (as indicated by their larger slope of accuracy as a function of epochs), we could see an increase in accuracy only after training for more epochs, as deeper models can provide more expressive representations but also introduce greater optimization challenges.

4.2 Effect of training duration

Duration of training has a strong effect on the performances of the models. We observe that training for a short duration of 10 epochs leads to lower accuracies and incomplete convergence of the network, which holds especially for deeper models. On the other hand, training for 30 epochs improves the accuracy of all models, especially for the four-block CNN. The accuracy curves show that the speed of improvement is slowing down and eventually reaches stability. This suggests that the longer training time can help the optimizer to really learn and adjust the parameters in the CNN model, which is very important when designing the experiment of the CNN, such as deciding how many epochs should we train the network in order to get a good balance between convergence and speed.

4.3 Effect of optimization strategy and learning rate

The comparisons between different optimization techniques in this subsection reveal that, in general, the performance of Adam and stochastic gradient descent is quite close to each other. The final peak accuracy of Adam is slightly higher and the convergence behavior is also the same when the models are trained for a large number of epochs. Batch size, number of iterations and learning rate are hyperparameters which may influence the stability and performance of training process. After trying various batch sizes and numbers of iterations, I got the best performance with batch size of 32 and 3000 iterations. This may be varied from case to case, as hyperparameters such as learning rate also may play important roles. Indeed, for learning rate = 0.001 gives better performance than learning rates less than 0.001 (training will be slow) and then learning rates greater than 0.001 (more oscillations) while in general it is advisable to use a moderate learning rate in CNN.

4.4 Conclusion

This project is about investigating the effects of architectural depth and training hyperparameters on various CNNs, with the goal of achieving the best classification accuracy on the CIFAR-10 image classification task. We will see how different choices of depth and training hyperparameters affect the performances of the designed models. As previously mentioned, the choice of optimization strategy and learning rate affects the training process, but to a lesser extent than the depth of the networks and the duration of training. The best configuration that we found is described in the next section, yielding a test accuracy of roughly 85%. It seems that more care must be taken when choosing the depth, length of training, and hyperparameters for CNNs if one wants to achieve the very best results on standard image classification benchmarks.

5 Reference

Lecun, Y., Bottou, L., Bengio, Y., & Haffner, P. (1998). Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, *86*(11), 2278–2324. https://doi.org/10.1109/5.726791

Krizhevsky, A., Sutskever, I., & Hinton, G. E. (2017). ImageNet classification with deep convolutional Neural Networks. *Communications of the ACM*, *60*(6), 84–90. https://doi.org/10.1145/3065386

He, K., Zhang, X., Ren, S., & Sun, J. (2016). Deep residual learning for image recognition. *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 770–778. https://doi.org/10.1109/cvpr.2016.90

Krizhevsky, A. (2009). *Learning Multiple Layers of Features from Tiny Images*. University of Toronto.

6 Potential issues and considerations

6.1 Experimental design

Experiments are conducted to study a variety of effects of different CNN configurations. The number of experiments is controlled by varying the depth of a network and the values of a few hyperparameters, including optimizers, learning rates, pooling operations and training lengths. A number of important hyperparameters are not explored to any great degree, and in hindsight, the number of experiments is relatively small, leaving a considerable subset of the design space of CNNs unexplored. No data augmentation techniques were applied in this study in order to isolate the effects of architectural depth and training hyperparameters on model performance.

6.2 Computational constraints and reproducibility

All experiments were run on CPU (due to hardware constraints) which imposed a constraint on the number of configurations/experiments that could be ran and number of times each experiment could be trained. Generally speaking, the longer models required more training for the performance to plateau. Although there is some randomness from initializations, shuffling of the mini-batches, our experiments show that the overall trends are stable. The numbers reported above are given assuming that one repeats the same training as described in the methodology below.

6.3 Reporting choices

We primarily focus the results on the test accuracy to demonstrate the generalization performance across the different configurations. The training behavior of the different models is shown using the accuracy plots and we refrain from presenting the loss curves and the variance over the different training runs to keep the visual presentation simple. Results for representative network configurations are given in the tables to highlight the main effects due to network depth and to hyperparameters.